

```

# =====
# THRESHOLD EFFECT IN EDGE DETECTION
# Canny Edge Detection with Different Threshold Values
# Google Colab
# =====

import cv2
import matplotlib.pyplot as plt

# -----
# STEP 1: Read the image uploaded in Colab Files
# -----

image_path = "/content/image1.jpg" # Change image1.jpg to your uploaded file name

img = cv2.imread(image_path)

# Check image
if img is None:
    print("Image not found. Check the file name in /content/")
else:

    # Convert BGR to RGB
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # Convert to grayscale
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # -----
    # STEP 2: Different Threshold Values
    # -----

    thresholds = [
        (50, 100),
        (100, 200),
        (150, 300),
        (200, 400)
    ]

    titles = [
        "Low Threshold (50, 100)",
        "Medium Threshold (100, 200)",
        "High Threshold (150, 300)",
        "Very High Threshold (200, 400)"
    ]

    # -----
    # STEP 3: Apply Canny Edge Detection
    # -----

    edges = []

    for low, high in thresholds:
        result = cv2.Canny(gray, low, high)
        edges.append(result)

    # -----
    # STEP 4: Display Original + Edge Detection Results
    # -----

```

```

plt.figure(figsize=(16, 9))

# Original image
plt.subplot(2, 3, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

# Threshold results
for i in range(4):
    plt.subplot(2, 3, i + 2)
    plt.imshow(edges[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()

# -----
# STEP 5: Save the Results
# -----

for (low, high), edge in zip(thresholds, edges):
    cv2.imwrite(
        f"/content/Canny_{low}_{high}.jpg",
        edge
    )

print("Edge detection completed successfully!")
print("Threshold values used: (50,100), (100,200), (150,300), (200,400)")

# -----
# STEP 6: Analysis
# -----

print("\n===== ANALYSIS =====")

print("\nLow Threshold (50,100):")
print("More edges are detected, but noise is also high.")

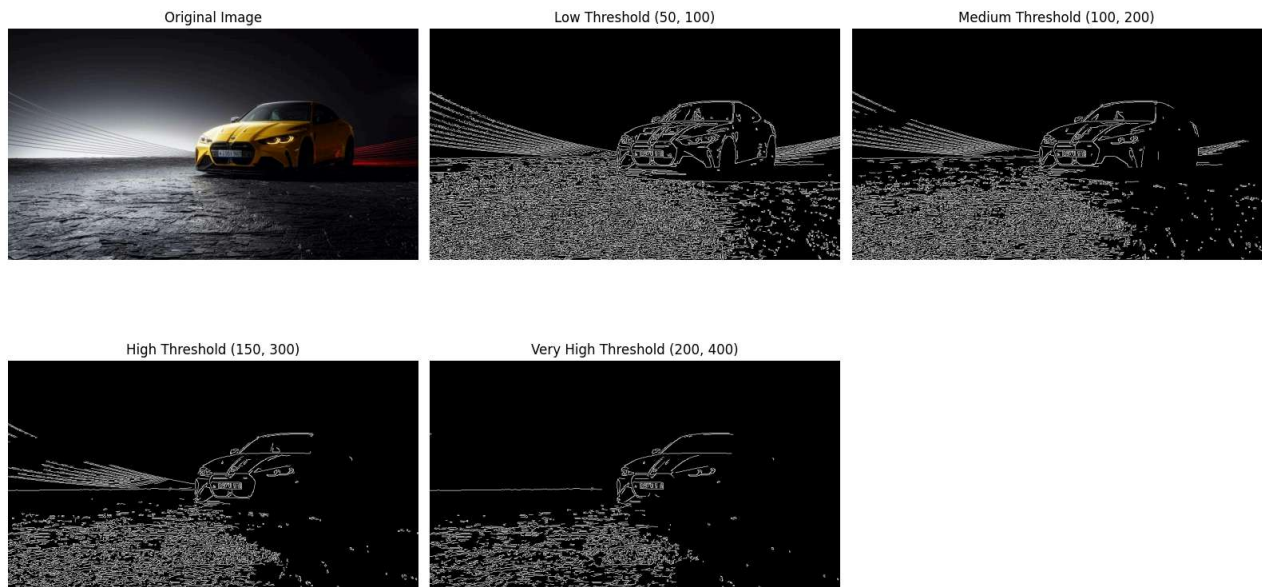
print("\nMedium Threshold (100,200):")
print("Good balance between edge continuity and noise reduction.")

print("\nHigh Threshold (150,300):")
print("Noise is reduced, but some weak edges may be lost.")

print("\nVery High Threshold (200,400):")
print("Only strong edges are detected and many weak edges may be missed.")

print("\n===== CONCLUSION =====")
print("Threshold selection affects edge continuity, accuracy, and noise.")
print("A suitable threshold provides accurate and continuous edges")
print("while reducing unwanted noise.")

```



Edge detection completed successfully!

Threshold values used: (50,100), (100,200), (150,300), (200,400)

===== ANALYSIS =====

Low Threshold (50,100):

More edges are detected, but noise is also high.

Medium Threshold (100,200):

Good balance between edge continuity and noise reduction.

High Threshold (150,300):

Noise is reduced, but some weak edges may be lost.

Very High Threshold (200,400):

Only strong edges are detected and many weak edges may be missed.

===== CONCLUSION =====

Threshold selection affects edge continuity, accuracy, and noise.

A suitable threshold provides accurate and continuous edges while reducing unwanted noise.